

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Input
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([
    Input(shape=(28,28,1)), # Explicitly define Input layer
    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Capture history for plotting
history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1)

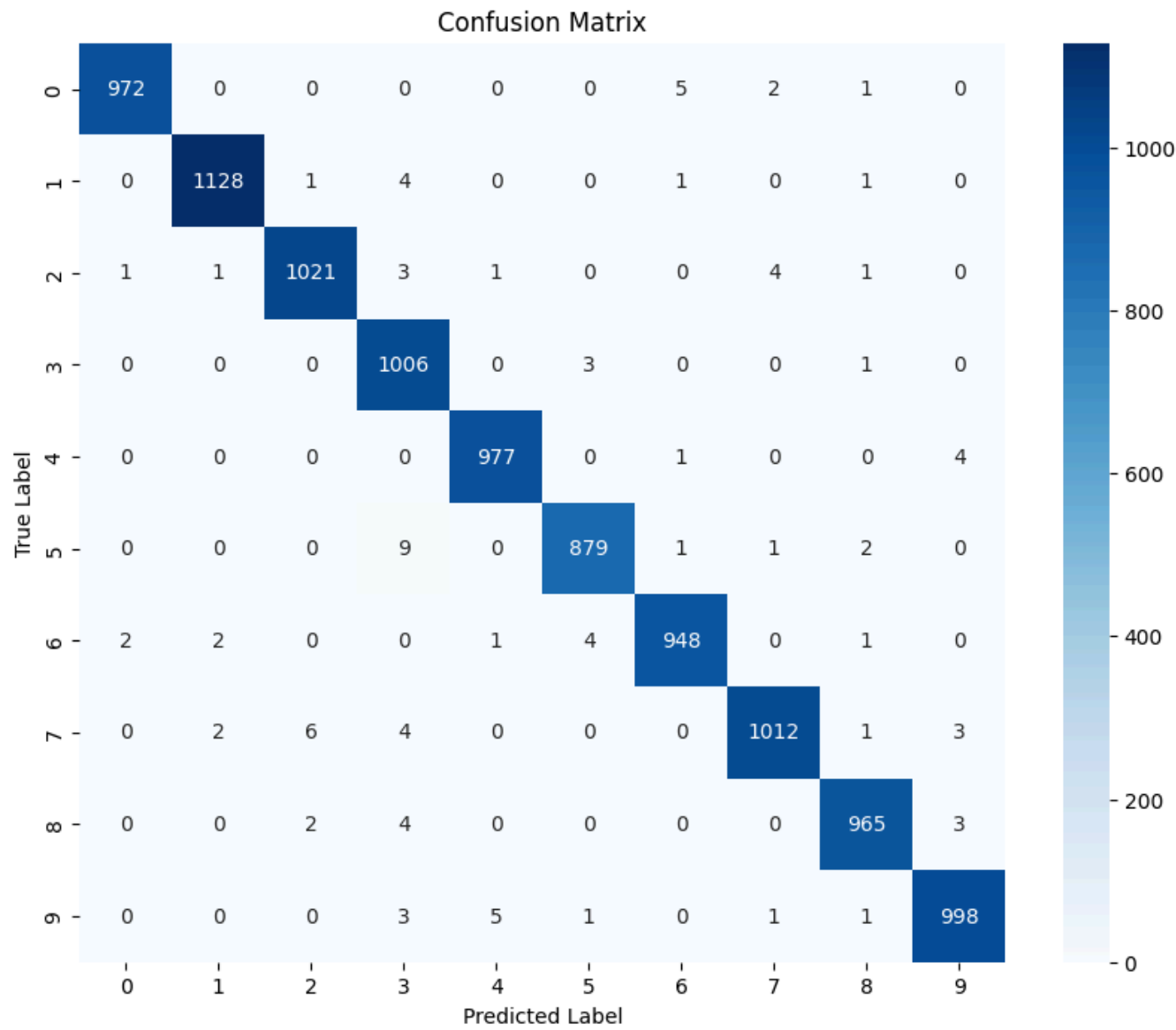
test_loss, test_acc = model.evaluate(X_test_cnn, y_test)
print("Test accuracy:", test_acc)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape` to `input`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
844/844 ━━━━━━━━━━━ 42s 49ms/step - accuracy: 0.9467 - loss: 0.1758 - val_accuracy: 0.9855 - val_loss: 0.0527
Epoch 2/10
844/844 ━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9841 - loss: 0.0499 - val_accuracy: 0.9860 - val_loss: 0.0447
Epoch 3/10
844/844 ━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9899 - loss: 0.0335 - val_accuracy: 0.9903 - val_loss: 0.0330
Epoch 4/10
844/844 ━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9919 - loss: 0.0245 - val_accuracy: 0.9913 - val_loss: 0.0305
Epoch 5/10
844/844 ━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9939 - loss: 0.0186 - val_accuracy: 0.9898 - val_loss: 0.0390
Epoch 6/10
844/844 ━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9952 - loss: 0.0149 - val_accuracy: 0.9925 - val_loss: 0.0246
Epoch 7/10
844/844 ━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9966 - loss: 0.0106 - val_accuracy: 0.9907 - val_loss: 0.0351
Epoch 8/10
844/844 ━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9965 - loss: 0.0101 - val_accuracy: 0.9912 - val_loss: 0.0421
Epoch 9/10
844/844 ━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9970 - loss: 0.0088 - val_accuracy: 0.9892 - val_loss: 0.0434
Epoch 10/10
844/844 ━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9976 - loss: 0.0065 - val_accuracy: 0.9900 - val_loss: 0.0446
313/313 ━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9906 - loss: 0.0382
Test accuracy: 0.9905999898910522
```

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# 2) Add Confusion Matrix
y_pred = model.predict(X_test_cnn)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
```



```
# 3) 2nd network: 1 conv, 1 pooling, 2 dense layers
model2 = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model2.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
print("--- Model 2 Summary ---")
model2.summary()

model2.fit(X_train_cnn, y_train, epochs=5, batch_size=64, validation_split=0.1)
```

--- Model 2 Summary ---

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape` to `Input` layers.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_1 (Flatten)	(None, 5408)	0
dense_2 (Dense)	(None, 64)	346,176
dense_3 (Dense)	(None, 10)	650

Total params: 347,146 (1.32 MB)
Trainable params: 347,146 (1.32 MB)
Non-trainable params: 0 (0.00 B)

Epoch 1/5
844/844 26s 29ms/step - accuracy: 0.9397 - loss: 0.2049 - val_accuracy: 0.9808 - val_loss: 0.0680

Epoch 2/5
844/844 30s 35ms/step - accuracy: 0.9808 - loss: 0.0662 - val_accuracy: 0.9857 - val_loss: 0.0552

Epoch 3/5
844/844 34s 27ms/step - accuracy: 0.9857 - loss: 0.0459 - val_accuracy: 0.9878 - val_loss: 0.0465

Epoch 4/5
844/844 22s 26ms/step - accuracy: 0.9895 - loss: 0.0336 - val_accuracy: 0.9825 - val_loss: 0.0562

Epoch 5/5
844/844 26s 31ms/step - accuracy: 0.9919 - loss: 0.0255 - val_accuracy: 0.9860 - val_loss: 0.0534

<keras.src.callbacks.history.History at 0x792bcb64f530>

```
# 4) 3rd network: 1 conv, 1 pooling, 1 dense layer
model3 = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(10, activation='softmax')
])

model3.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
print("--- Model 3 Summary ---")
model3.summary()
```

```
model3.fit(X_train_cnn, y_train, epochs=5, batch_size=64, validation_split=0.1)
```

--- Model 3 Summary ---
Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_3 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_2 (Flatten)	(None, 5408)	0
dense_4 (Dense)	(None, 10)	54,090

Total params: 54,410 (212.54 KB)
Trainable params: 54,410 (212.54 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/5
844/844 22s 25ms/step - accuracy: 0.9191 - loss: 0.2920 - val_accuracy: 0.9708 - val_loss: 0.1099
Epoch 2/5
844/844 19s 23ms/step - accuracy: 0.9702 - loss: 0.1065 - val_accuracy: 0.9795 - val_loss: 0.0790
Epoch 3/5
844/844 21s 25ms/step - accuracy: 0.9784 - loss: 0.0759 - val_accuracy: 0.9812 - val_loss: 0.0676
Epoch 4/5
844/844 20s 24ms/step - accuracy: 0.9815 - loss: 0.0625 - val_accuracy: 0.9817 - val_loss: 0.0635
Epoch 5/5
844/844 21s 25ms/step - accuracy: 0.9841 - loss: 0.0531 - val_accuracy: 0.9825 - val_loss: 0.0642

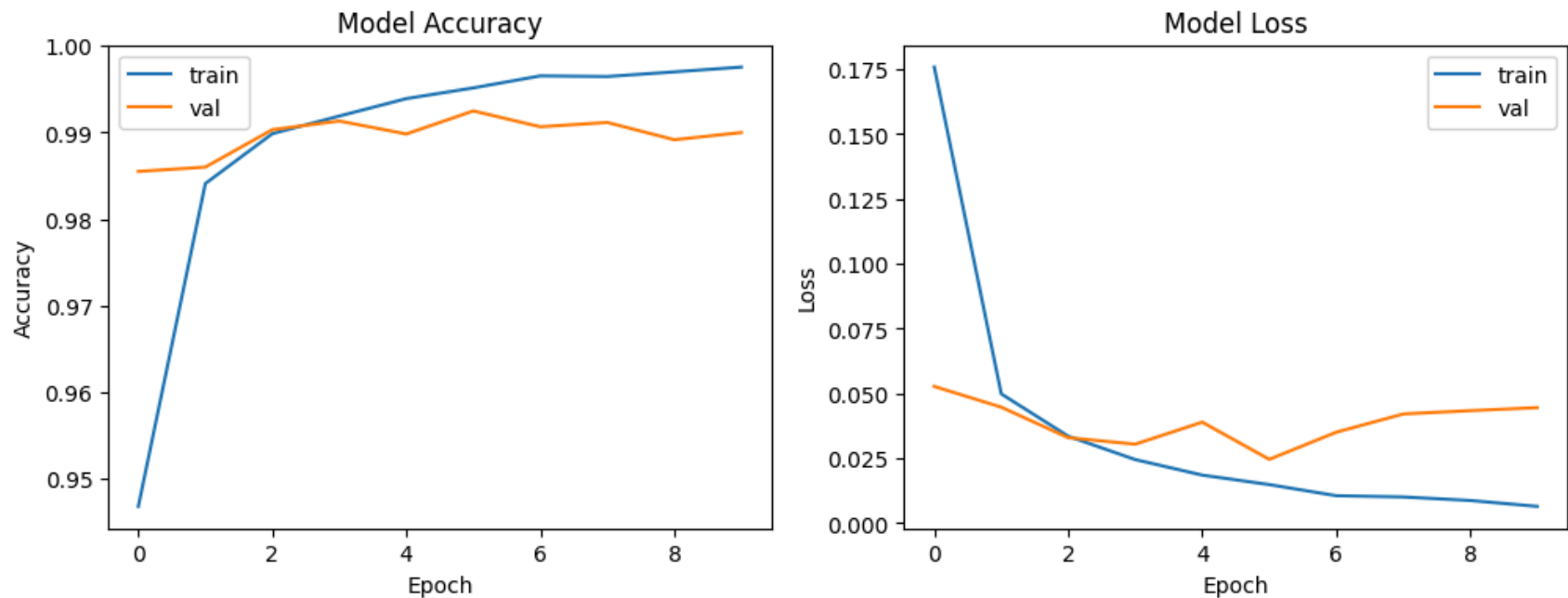
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# 1) Accuracy vs Loss Plot
def plot_history(history):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(history.history['accuracy'], label='train')
    ax1.plot(history.history['val_accuracy'], label='val')
    ax1.set_title('Model Accuracy')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Accuracy')
    ax1.legend()

    ax2.plot(history.history['loss'], label='train')
    ax2.plot(history.history['val_loss'], label='val')
    ax2.set_title('Model Loss')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Loss')
    ax2.legend()
    plt.show()

# Call the function for the first model
plot_history(history)
```



```
from google.colab import files
import cv2
from PIL import Image
import io

# Upload images from your desktop
print("Please upload your handwritten images (0-9)...")
uploaded = files.upload()

plt.figure(figsize=(15, 5))

for i, filename in enumerate(uploaded.keys()):
    # Read and preprocess the image
    image_data = uploaded[filename]
    img = Image.open(io.BytesIO(image_data)).convert('L') # Convert to grayscale
    img = img.resize((28, 28)) # Resize to 28x28

    # Convert to array and normalize
    img_array = np.array(img)
```

```
# MNIST images have black backgrounds and white digits.
# If your drawing is black ink on white paper, we need to invert it.
# We assume inversion is needed if the center pixel is brighter than the corner.
if img_array[0,0] > 127:
    img_array = 255 - img_array

img_array = img_array / 255.0
img_input = img_array.reshape(1, 28, 28, 1)

# Predict using the first model
prediction = model.predict(img_input)
predicted_digit = np.argmax(prediction)
# Plot result
plt.subplot(1, len(uploaded), i + 1)
plt.imshow(img_array, cmap='gray')
plt.title(f"Pred: {predicted_digit}")
plt.axis('off')

plt.show()
```

Please upload your handwritten images (0-9)...

Choose Files

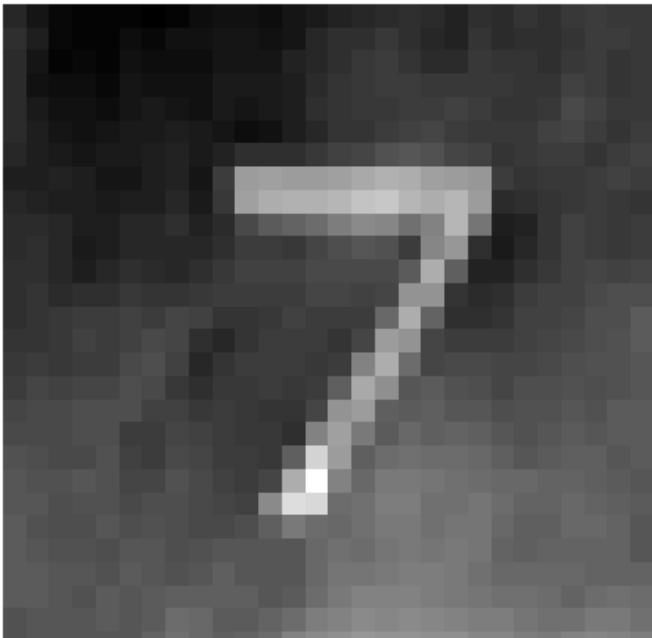
No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving WhatsApp Image 2026-04-27 at 3.02.12 AM.jpeg to WhatsApp Image 2026-04-27 at 3.02.12 AM (1).jpeg

1/1 0s 73ms/step

Pred: 1



Start coding or [generate](#) with AI.